

METRICMIND

Agentic Semantic BI Engine

FINAL PROJECT REPORT

PREMIUM EVIDENCE EDITION

Architecture • UI • APIs • Testing • Git Evidence

AXLERO SOLUTIONS

QA • API TESTING • DOCUMENTATION

AUGUST 2026

01

EXECUTIVE OVERVIEW

A concise project snapshot with verified testing status.

PROJECT	FRONTEND	BACKEND	DATABASE
MetricMind	React + Vite	FastAPI	MySQL

Executive Finding

MetricMind combines a business dashboard, API layer, authentication flow, analytics modules and governed business-metric concepts. Recorded manual testing shows strong authentication/UI coverage, with open HTTP 500 issues in Dashboard, Reports and Analytics APIs.

APIS TESTED	PASSED	FAILED	OPEN BUGS
14	8	6	3 HIGH

- No business numbers are invented in this report.
- Evidence pages use screenshots captured during the project workflow.
- Sensitive tokens/credentials are redacted in the report.

02

PROBLEM & OBJECTIVES

Why governed conversational analytics matters.

Traditional business analytics can require knowledge of databases, SQL and complex reporting systems. Direct text-to-SQL can also produce incorrect joins, inconsistent business metrics and uncontrolled access to data.

Core Requirement

The LLM/agent should interact with the semantic layer rather than directly writing unrestricted SQL.

- Interactive business analytics dashboard.
- Sales, revenue, orders, profit and KPI views.
- Reports and analytical views.
- Natural-language business interaction.
- Governed metrics instead of unrestricted SQL access.
- Transparent API responses and trace information where applicable.
- Responsive and user-friendly UI.

03

ARCHITECTURE

Controlled flow from question to answer.

01	USER QUESTION
02	FRONTEND / UI
03	AGENT / ORCHESTRATOR
04	SEMANTIC / METRIC LAYER
05	DATABASE / DATA
06	ANSWER + CHART + API TRACE

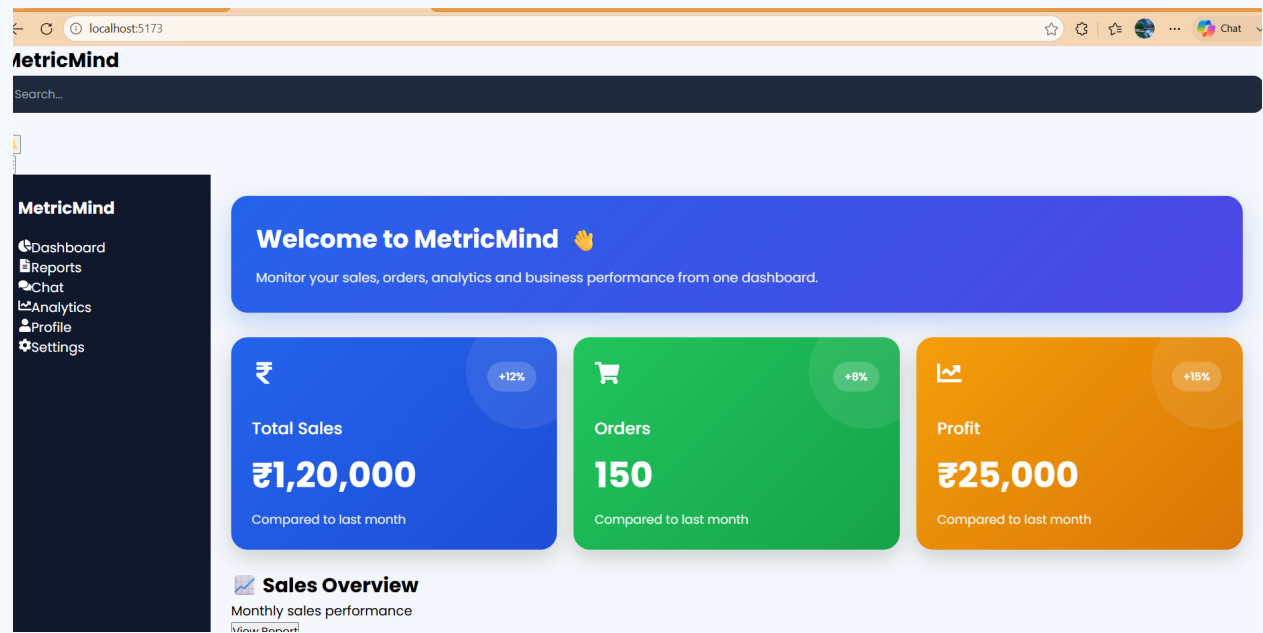
Architecture Note

The implemented repository is documented as React + Vite, FastAPI and MySQL. The final report records this implementation rather than claiming an unimplemented stack.

04 FRONTEND & UI EVIDENCE

Real application screenshots from the project.

Dashboard — main navigation, welcome panel and KPI cards



Sales Overview — monthly sales chart and recent transactions



05

AUTHENTICATION & API EVIDENCE

Swagger/OpenAPI evidence with sensitive values redacted.

Protected profile endpoint — successful 200 response

The screenshot displays the Swagger UI interface for a REST API. The top section, titled 'Responses', shows the details of a 200 status code response. The 'Request URL' is `http://127.0.0.1:8080/profile`. The 'Server response' section shows the 'Response body' as a JSON object: `{ "id": 0, "name": "string", "email": "user@example.com" }`. The 'Response headers' section shows: `content-length: 63, content-type: application/json, date: Sat, 01 Aug 2026 16:16:14 GMT, server: uvicorn`. Below this, a table lists the response with a 'Code' of 200 and a 'Description' of 'Successful Response'. The 'Media type' is set to 'application/json'. The 'Example Value' section shows the same JSON object as the response body. The 'Links' column indicates 'No links'.

Swagger endpoint catalog — Reports and Analytics routes

The screenshot displays the Swagger UI interface for a REST API, showing the endpoint catalog. The top section, titled 'Responses', shows the details of a 200 status code response for the '/profile' endpoint. Below this, the 'Endpoint catalog' section lists the following endpoints:

- PUT /profile** Update Profile
- GET /reports/sales** Sales Report
- GET /reports/revenue** Revenue Report
- GET /reports/customer** Customer Report
- GET /reports/monthly** Monthly Report
- GET /analytics** Analytics
- GET /analytics/sales** Analytics Sales
- GET /analytics/products** Analytics Products

The 'Reports' and 'Analytics' sections are expanded, showing the details of each endpoint. The 'Reports' section shows the following endpoints:

- GET /reports/sales** Sales Report
- GET /reports/revenue** Revenue Report
- GET /reports/customer** Customer Report
- GET /reports/monthly** Monthly Report

The 'Analytics' section shows the following endpoints:

- GET /analytics** Analytics
- GET /analytics/sales** Analytics Sales
- GET /analytics/products** Analytics Products

Sensitive authorization values are intentionally redacted in the report.

06

RUNTIME EVIDENCE

Frontend and backend servers were brought up during testing.

Frontend server — Vite ready on localhost:5173

```
PS C:\Users\prash\OneDrive\Desktop\MetricMind\frontend> npm run dev

> frontend@0.0.0 dev
> vite

VITE v8.2.0 ready in 1561 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
PS C:\Users\prash\OneDrive\Desktop\MetricMind\frontend>
```

Backend server — FastAPI startup and Swagger/OpenAPI 200 responses

```
PS C:\Users\prash\OneDrive\Desktop\MetricMind\backend>
ain:app --reload
icmind
2026-08-03 12:17:29,180 INFO sqlalchemy.engine.Engine SELECT DATABASE()
2026-08-03 12:17:29,180 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-08-03 12:17:29,196 INFO sqlalchemy.engine.Engine SELECT @@sql_mode
2026-08-03 12:17:29,196 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-08-03 12:17:29,201 INFO sqlalchemy.engine.Engine SELECT @@lower_case_tabl
e_names
2026-08-03 12:17:29,201 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-08-03 12:17:29,205 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-08-03 12:17:29,207 INFO sqlalchemy.engine.Engine DESCRIBE `metricmind`.`u
sers`
2026-08-03 12:17:29,207 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-08-03 12:17:29,423 INFO sqlalchemy.engine.Engine COMMIT
INFO: Started server process [24748]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:62265 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:62265 - "GET /openapi.json HTTP/1.1" 200 OK
```

07

AGENTIC WORKFLOW & SEMANTIC LAYER

Business question → governed analysis → answer.

Stage	What happens
Question	User asks a natural-language business question.
Agent	Intent and required analysis are identified.
Metric Catalog	Governed metrics/dimensions define business meaning.
Semantic Query	The analytical request is expressed through the governed layer.
Data Result	Controlled result is returned.
Final Answer	Business-friendly explanation is produced.
Chart / Trace	Visual and API trace information is shown where supported.

Root-Cause Example

“Why did our European margins drop last quarter?” → margin query → detect significant change → cost breakdown → shipping/material/other drivers → root-cause explanation.

08

GOVERNANCE

Explicit safeguards for the analytics workflow.

05	1,000	NO RAW SQL
MAX QUERIES / TURN	MAX ROWS / QUERY	AGENT ACCESS RULE

- Maximum 5 queries per turn.
- Maximum 1,000 rows per query.
- No unrestricted SQL from the agent.
- Semantic-layer-only / governed data access.
- API trace where applicable.

09

TESTING EVIDENCE

Recorded manual API and UI testing results.

Area	Result
Backend Server	PASS
Frontend Server	PASS
Authentication	PASS
Register / Login / Profile	PASS
Home APIs	PASS
Dashboard UI / KPI / Sales Chart	PASS
Home Dashboard API	FAIL — HTTP 500
Reports APIs	FAIL — HTTP 500
Analytics APIs	FAIL — HTTP 500

Recorded Reports API failure — HTTP 500

Execute

Clear

Responses

Curl

curl -X 'GET' \nhttps://127.0.0.1:8080/reports/sales' \n-H 'accept: application/json'

Request URL

http://127.0.0.1:8080/reports/sales

Server response

Code

Details

500

Error: Internal Server Error

undocumented

Response body

Internal Server Error

Download

Response headers

content-length: 21\ncontent-type: text/plain; charset=utf-8\ndate: Sat, 01 Aug 2026 10:47:58 GMT\nserver: uvicorn

Responses

Code

Description

Links

200

Successful Response

No links

Media type

application/json

Controls

Accept header

Example Value

Schema

"string"

Testing Status

Partially Passed — authentication and core UI areas worked; backend Dashboard/Reports/Analytics issues remain open.

BUG REPORT

Open issues documented from the test session.

Bug ID	Module	Priority	Status
BUG-001	Dashboard	High	Open
BUG-002	Reports	High	Open
BUG-003	Analytics	High	Open

- Home Dashboard API returns HTTP 500 Internal Server Error.
- Reports APIs return HTTP 500 Internal Server Error.
- Analytics APIs return HTTP 500 Internal Server Error.

These issues require developer investigation followed by regression testing before production deployment.

11

DOCUMENTATION & EVIDENCE PACK

The final repository documentation is supported by an evidence folder.

Document / Evidence	Purpose
README.md	Project overview, setup, structure and features
API.md	REST API documentation and Swagger/OpenAPI reference
API_TESTING.md	Manual API test record
TESTING.md	Testing scope, environment and results
BUG_REPORT.md	Open defect register
INSTALLATION.md	Installation and startup instructions
USER_GUIDE.md	Application usage guide
docs/screenshots/	UI and API evidence screenshots

Evidence Principle

Only screenshots from the final tested version should be used in the submission. The report separates verified evidence from planned or unsupported claims.

12

TEAM & GITHUB EVIDENCE

Contribution and version-control traceability.

Member	Contribution
Shyam	Team Lead, Daily Task Distribution, DevOps, Integration
Chetan	Backend / Agent Orchestration
Mansi	Data / Database / Semantic Validation
Yoshita	UI / Frontend
Himesh	Frontend / Reassigned Work
Prashant	QA, API Testing, Documentation

Repository evidence

```
PS C:\Users\prash\OneDrive\Desktop\MetricMind> git push origin docs
info: please complete authentication in your browser...
Enumerating objects: 267, done.
Counting objects: 100% (206/206), done.
Delta compression using up to 8 threads
Compressing objects: 100% (87/87), done.
Writing objects: 100% (138/138), 1.48 MiB | 625.00 KiB/s, done.
Total 138 (delta 58), reused 81 (delta 46), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (58/58), completed with 20 local objects.
To https://github.com/ShyamVarmora/MetricMind.git
 67f5895..e179602 docs -> docs
PS C:\Users\prash\OneDrive\Desktop\MetricMind>
PS C:\Users\prash\OneDrive\Desktop\MetricMind>
```

13

LIMITATIONS & TECHNOLOGY NOTES

Transparent record of implementation differences.

The project specification describes Next.js, Cube.dev and PostgreSQL, while the repository documentation used for this final report describes React + Vite, FastAPI and MySQL.

Technology Substitution

The report records the implemented stack and does not present specification-only technologies as completed features.

The report also avoids inventing revenue, regional or root-cause numbers. Any such values should be taken from verified application output.

Open backend errors should be fixed and regression-tested before production deployment.

14

FINAL ASSESSMENT

Submission-ready conclusion.

METRICMIND

Agentic Semantic BI Engine

MetricMind demonstrates a governed conversational analytics approach by combining a web interface, backend APIs, business metrics, database access, visualization and test evidence.

Authentication and core UI areas were successfully tested. Dashboard, Reports and Analytics backend errors remain open and should be resolved before production deployment.

FINAL PROJECT REPORT • AUGUST 2026